

qwen_saeed_agent_stargazer

June 10, 2026

1 Qwen + Saeed traj-eval Agent on a Real STARGAZER Finding Task

This notebook tests the Saeed traj-eval planner, engineer, reviewer, and executor flow with Qwen on a real STARGAZER-style radial-velocity task.

Workflow boundary: the STARGAZER result must be produced by the agent workflow itself. The planner receives only a sanitized observation summary, the engineer writes a complete Python file, one reviewer agent checks the code before execution and the executed result afterward, the executor only runs the approved file, and the hidden-truth STARGAZER benchmark runs only after an executor-produced submission exists.

The external repo is read-only:

C:\Dev\src\github.com\msaeedmt\traj-eval

All generated files are written here:

C:\Users\Anwender\Science-Work-Flow-\outputs\qwen_saeed_agent_stargazer

```
[1]: from pathlib import Path
import ast
import json
import os
import shutil
import subprocess
import sys
import time

import pandas as pd
from IPython.display import Markdown, display

SCIENCE_ROOT = Path(r"C:\Users\Anwender\Science-Work-Flow-")
SAEED_REPO = Path(r"C:\Dev\src\github.com\msaeedmt\traj-eval")
SAEED_SRC = SAEED_REPO / "src"
OUT = SCIENCE_ROOT / "outputs" / "qwen_saeed_agent_stargazer"
OUT.mkdir(parents=True, exist_ok=True)

TASK_RUN = SCIENCE_ROOT / "outputs" / "notebook_qwen_minimal_stargazer_task1" / "runs" / "stargazer_real_real_001_minimal"
```

```

OBS_PATH = TASK_RUN / "stargazer_observations.json"

for path in [SAEED_REPO, SAEED_SRC, OBS_PATH]:
    assert path.exists(), f"Missing required path: {path}"

if str(SAEED_SRC) not in sys.path:
    sys.path.insert(0, str(SAEED_SRC))
SCIENCE_SRC = SCIENCE_ROOT / "src"
if str(SCIENCE_SRC) not in sys.path:
    sys.path.insert(0, str(SCIENCE_SRC))

pd.set_option("display.max_colwidth", 160)
pd.set_option("display.width", 180)

def run_command(args, cwd=None, env=None, timeout=120):
    start = time.time()
    proc = subprocess.run(
        args,
        cwd=str(cwd) if cwd else None,
        env=env,
        text=True,
        capture_output=True,
        timeout=timeout,
    )
    return {
        "args": [str(a) for a in args],
        "cwd": str(cwd) if cwd else None,
        "returncode": proc.returncode,
        "elapsed_seconds": round(time.time() - start, 3),
        "stdout": proc.stdout,
        "stderr": proc.stderr,
    }

def write_json(path: Path, payload):
    path.write_text(json.dumps(payload, indent=2), encoding="utf-8")

display(Markdown(f"Real STARGAZER observation file: `{OBS_PATH}`"))

```

Real STARGAZER observation file: C:\Users\Anwender\Science-Work-Flow-\outputs\notebook_qwen_minim

1.1 1. Read-only Saeed Repo Preflight

The agent repo must stay clean. This notebook only imports its role constructors.

```

[2]: pre_branch = run_command(["git", "branch", "--show-current"], cwd=SAEED_REPO)
pre_status = run_command(["git", "status", "--short"], cwd=SAEED_REPO)
preflight = {
    "branch": pre_branch["stdout"].strip(),

```

```

    "status_short": pre_status["stdout"].strip(),
    "repo_untouched_before": pre_status["stdout"].strip() == "",
}
(OUT / "finding_repo_preflight.json").write_text(json.dumps(preflight,
    ↪indent=2), encoding="utf-8")
display(pd.DataFrame([[k, v] for k, v in preflight.items()], columns=["Field",
    ↪"Value"])))
assert preflight["repo_untouched_before"], "Saeed repo is dirty before the run."

```

```

           Field Value
0           branch  Han
1      status_short
2 repo_untouched_before  True

```

1.2 2. Load Qwen Environment Without Printing Secrets

The API key is loaded but never displayed.

[3]: ENV_PATH = SCIENCE_ROOT / "configs" / "cmbagent_eval" / "provider.local.env"

```

def load_env_file(path: Path):
    loaded = {}
    for line in path.read_text(encoding="utf-8").splitlines():
        text = line.strip()
        if not text or text.startswith("#") or "=" not in text:
            continue
        key, value = text.split("=", 1)
        key = key.strip()
        value = value.strip().strip('"').strip("'")
        os.environ[key] = value
        loaded[key] = value
    return loaded

load_env_file(ENV_PATH)
base_url = os.getenv("OPENAI_BASE_URL") or os.getenv("OPENAI_API_BASE")
model = os.getenv("CMBAGENT_EVAL_LOCAL_MODEL") or "openai/Qwen3.5-27B-Q5_K_M.
    ↪gguf"
os.environ["OPENAI_BASE_URL"] = base_url
os.environ["TRAJ_EVAL_MODEL"] = model

display(pd.DataFrame(
    [
        ["Base URL", base_url],
        ["Model", model],
        ["API key set", bool(os.getenv("OPENAI_API_KEY"))],
    ],
    columns=["Field", "Value"],
))

```

```
assert os.getenv("OPENAI_API_KEY"), "OPENAI_API_KEY missing."
```

	Field	Value
0	Base URL	http://131.220.150.238:8080/v1
1	Model	openai/Qwen3.5-27B-Q5_K_M.gguf
2	API key set	True

1.3 3. Qwen Provider Probe With Thinking Enabled

This proves provider reachability before testing the agent route.

```
[ ]: from openai import OpenAI

client = OpenAI(base_url=base_url, api_key=os.getenv("OPENAI_API_KEY"))
probe_response = client.chat.completions.create(
    model=model,
    messages=[{"role": "user", "content": "Think briefly, then reply with OK."}],
    temperature=0,
    max_tokens=128,
    extra_body={"chat_template_kwargs": {"enable_thinking": True}},
)
msg = probe_response.choices[0].message
reasoning_text = getattr(msg, "reasoning_content", None) or ""
probe = {
    "ok": bool((msg.content or "").strip() or reasoning_text.strip()),
    "finish_reason": probe_response.choices[0].finish_reason,
    "content": msg.content,
    "reasoning_present": bool(reasoning_text),
    "qwen_thinking_enabled": True,
}
(OUT / "finding_qwen_probe.json").write_text(json.dumps(probe, indent=2),
    encoding="utf-8")
display(pd.DataFrame([[k, v] for k, v in probe.items()], columns=["Field",
    "Value"]))
assert probe["ok"], "Qwen probe failed."
```

	Field	Value
0	ok	True
1	finish_reason	length
2	content	
3	reasoning_present	True
4	qwen_thinking_enabled	True

1.4 4. Inspect the Real Observation Task

These are sanitized observations only. No hidden truth is loaded for the finding task.

```

[5]: observations = json.loads(OBS_PATH.read_text(encoding="utf-8"))
if isinstance(observations, dict) and "observations" in observations:
    rows = observations["observations"]
elif isinstance(observations, list):
    rows = observations
else:
    rows = observations.get("data", [])

obs_df = pd.DataFrame(rows)
numeric_cols = [c for c in obs_df.columns if pd.api.types.
    ↳is_numeric_dtype(obs_df[c])]
time_col = next((c for c in ["times_days", "time_days", "time", "t", "jd"] if c
    ↳in obs_df.columns), numeric_cols[0])
rv_col = next((c for c in ["rvs_ms", "rv_ms", "rv", "radial_velocity",
    ↳"velocity"] if c in obs_df.columns), None)
sigma_col = next((c for c in ["sigmas_ms", "sigma_ms", "rv_error", "error",
    ↳"uncertainty"] if c in obs_df.columns), None)
inst_col = next((c for c in ["instruments", "instrument", "inst",
    ↳"instrument_id"] if c in obs_df.columns), None)

task_summary = {
    "task_label": "real_001 sanitized STARGAZER observations",
    "n_observations": int(len(obs_df)),
    "columns": list(obs_df.columns),
    "time_column": time_col,
    "rv_column": rv_col,
    "sigma_column": sigma_col,
    "instrument_column": inst_col,
    "json_shape": "top-level dict with task_id and observations; observations
    ↳is a dict of parallel arrays",
    "observation_array_columns": ["times_days", "rvs_ms", "sigmas_ms",
    ↳"instruments"],
    "baseline_days": float(obs_df[time_col].max() - obs_df[time_col].min()) if
    ↳time_col in obs_df else None,
    "instrument_count": int(obs_df[inst_col].nunique()) if inst_col else None,
}
(OUT / "finding_task_summary.json").write_text(json.dumps(task_summary,
    ↳indent=2), encoding="utf-8")
display(pd.DataFrame([[k, v] for k, v in task_summary.items()],
    ↳columns=["Field", "Value"]))
display(obs_df.head())

```

	Field	Value
0	task_label	real_001 sanitized STARGAZER observations

```

1          n_observations                                ↪
↪          639
2          columns                                     ↪
↪[instruments, rvs_ms, sigmas_ms, times_days]
3          time_column                                ↪
↪          times_days
4          rv_column                                  ↪
↪          rvs_ms
5          sigma_column                               ↪
↪          sigmas_ms
6          instrument_column                           ↪
↪          instruments
7          json_shape  top-level dict with task_id and observations;↪
↪observations is a dict of parallel arrays
8  observation_array_columns                           ↪
↪[times_days, rvs_ms, sigmas_ms, instruments]
9          baseline_days                               ↪
↪          7237.393404
10         instrument_count                             ↪
↪          6

instruments  rvs_ms  sigmas_ms  times_days
0  inst_A    -6.0      9.0  2.449611e+06
1  inst_A    27.0      9.0  2.449612e+06
2  inst_A   -20.0      7.0  2.449655e+06
3  inst_A   -58.0      7.0  2.449728e+06
4  inst_A     4.0      7.0  2.449729e+06

```

1.5 5. Agent Workflow Contract

No planet parameters are computed before the engineer script executes. The only pre-agent payload is a sanitized observation summary; the engineer must write a complete Python program that reads `stargazer_observations.json` and writes `agent_submission.json` in its execution directory.

```

[6]: AGENT_WORK = OUT / "agent_workflow"
if AGENT_WORK.exists():
    shutil.rmtree(AGENT_WORK)
AGENT_WORK.mkdir(parents=True, exist_ok=True)

AGENT_OBS_COPY = AGENT_WORK / "stargazer_observations.json"
shutil.copy2(OBS_PATH, AGENT_OBS_COPY)

agent_contract = {
    "input_file": "stargazer_observations.json",
    "output_file": "agent_submission.json",
    "hidden_truth_available_to_agents": False,
    "engineer_output_contract": [
        "Python code only",
    ]
}

```

```

        "read stargazer_observations.json from the current working directory",
        "write agent_submission.json in the current working directory",
        "do not read truth, benchmark, task, answer, evaluator, or real_001_
↳files",
        "do not import or call external precomputed inference modules",
        "include numerical search logic inside the script",
    ],
    "executor_contract": [
        "never edits code",
        "runs the exact reviewer-approved iteration script",
        "records command, script path, exit code, stdout, stderr, generated_
↳files, and parsed submission",
    ],
    "reviewer_modes": ["code_review", "result_review"],
}
write_json(OUT / "agent_workflow_contract.json", agent_contract)
display(pd.DataFrame([[k, v] for k, v in agent_contract.items()],
↳columns=["Contract field", "Value"]))

```

```

Contract field \
0         input_file
1         output_file
2 hidden_truth_available_to_agents
3     engineer_output_contract
4         executor_contract
5         reviewer_modes

↳
↳Value
0
↳
↳stargazer_observations.
↳json
1
↳
↳agent_submission.
↳json
2
↳
↳False
3 [Python code only, read stargazer_observations.json from the current working_
↳directory, write agent_submission.json in the current working directory, do_
↳not...
4 [never edits code, runs the exact reviewer-approved iteration script, records_
↳command, script path, exit code, stdout, stderr, generated files, and parsed s...
5
↳
↳[code_review,
↳result_review]

```

1.6 6. Load Saeed Agent Role Prompts and Call Qwen With Thinking Enabled

With Qwen thinking enabled, AG2 can lose the visible answer because Qwen may return content in provider-specific reasoning fields. To preserve the Saeed role identities while keeping outputs parseable, this notebook loads Saeed role definitions and calls the OpenAI-compatible Qwen endpoint directly.

```
[7]: from traj_eval.agents.roles import (
    CRITIC_SYSTEM_MESSAGE as SAEED_CRITIC_SYSTEM_MESSAGE,
    ENGINEER_SYSTEM_MESSAGE as SAEED_ENGINEER_SYSTEM_MESSAGE,
    PLANNER_SYSTEM_MESSAGE as SAEED_PLANNER_SYSTEM_MESSAGE,
)

PLANNER_SYSTEM_MESSAGE = (
    SAEED_PLANNER_SYSTEM_MESSAGE
    + "\nYou are the planner. Plan only. Use numbered steps. Do not solve or
    ↪provide planet parameters."
)

ENGINEER_SYSTEM_MESSAGE = (
    SAEED_ENGINEER_SYSTEM_MESSAGE
    + "\nYou are the engineer. Return Python code only. No markdown fences, no
    ↪prose."
)

REVIEWER_SYSTEM_MESSAGE = (
    SAEED_CRITIC_SYSTEM_MESSAGE
    + "\nYou are the single reviewer/critic for both code_review and
    ↪result_review modes. "
    "Judge software safety, schema compliance, scientific plausibility, and
    ↪numerical validity. "
    "Use the planner plan as review evidence when it is provided. Keep
    ↪numerical criteria stable across iterations; do not reverse a prior
    ↪criterion unless new evidence is explicit. "
    "Return a verdict line first, then short evidence."
)

qwen_trace = []

def extract_qwen_text(message):
    content = (getattr(message, "content", None) or "").strip()
    reasoning = (getattr(message, "reasoning_content", None) or "").strip()
    if content:
        return content
    if reasoning:
        return reasoning
    extra = getattr(message, "model_extra", None) or {}
    provider = extra.get("provider_specific_fields") or {}
    return (provider.get("reasoning_content") or "").strip()
```



```

def qwen_role_call(role_name: str, system_message: str, user_message: str,
    ↪max_tokens: int = 1200, phase: str | None = None, iteration: int | None =
    ↪None, enable_thinking: bool = True):
    response = client.chat.completions.create(
        model=model,
        messages=[
            {"role": "system", "content": system_message},
            {"role": "user", "content": user_message},
        ],
        temperature=0,
        max_tokens=max_tokens,
        extra_body={"chat_template_kwargs": {"enable_thinking":
    ↪enable_thinking}},
    )
    message = response.choices[0].message
    content_text = (getattr(message, "content", None) or "").strip()
    reasoning_text = (getattr(message, "reasoning_content", None) or "").strip()
    record = {
        "role": role_name,
        "phase": phase,
        "iteration": iteration,
        "finish_reason": response.choices[0].finish_reason,
        "text": extract_qwen_text(message),
        "content_present": bool(content_text),
        "reasoning_present": bool(reasoning_text),
        "reasoning_excerpt": reasoning_text[:800],
        "tokens": response.usage.model_dump() if response.usage else {},
        "enable_thinking": enable_thinking,
    }
    qwen_trace.append(record)
    return record

display(Markdown(
    "Loaded Saeed planner, engineer, and critic role definitions. The critic
    ↪identity is used once as a reviewer agent with `code_review` and
    ↪`result_review` modes."
))

```

Loaded Saeed planner, engineer, and critic role definitions. The critic identity is used once as a reviewer agent with `code_review` and `result_review` modes.

1.7 7. Saeed/Qwen Planner: Real Finding Task

The planner receives only the sanitized observation summary and the workflow contract. It does not receive hidden truth, benchmark files, precomputed planets, or any inferred parameters.

```
[8]: planner_prompt = f'''
Plan a real STARGAZER radial-velocity finding task.

Sanitized observation summary:
{json.dumps(task_summary, indent=2)}

Workflow contract:
{json.dumps(agent_contract, indent=2)}

Goal:
Create a plan for an engineer who must write a standalone Python script. The
    ↳ script will read stargazer_observations.json, perform its own numerical
    ↳ search for plausible planet signals, and write agent_submission.json.

Plan only. Do not evaluate against hidden truth. Do not invent planet
    ↳ parameters.
'''

planner_record = qwen_role_call("planner", PLANNER_SYSTEM_MESSAGE,
    ↳ planner_prompt, max_tokens=900, phase="planning", iteration=0)
planner_summary = planner_record["text"].strip()
assert planner_summary, "Planner returned no visible content."
write_json(OUT / "planner_record.json", planner_record)
(OUT / "planner_summary.txt").write_text(planner_summary, encoding="utf-8")
display(Markdown(planner_summary))
```

Thinking Process:

1. Analyze the Request:

- **Role:** PLANNER in a multi-agent scientific-reasoning team.
- **Task:** Create a short, ordered plan (max 5 steps) for a real STARGAZER radial-velocity finding task.
- **Input:** Sanitized observation summary (639 observations, 6 instruments, columns: times_days, rvs_ms, sigmas_ms, instruments).
- **Constraints:**
 - Output a numbered list of concrete steps.
 - Max 5 steps.
 - Do NOT solve, compute, or write final code.
 - Do NOT provide planet parameters.
 - Be explicit about which step verifies the result.
 - Follow the workflow contract (input/output files, no external truth, numerical search logic inside script).
- **Goal:** Plan for an engineer to write a standalone Python script that reads stargazer_observations.json, performs numerical search, and writes agent_submission.json.

2. Determine the Steps:

- Step 1: Data Loading & Preprocessing. The engineer needs to load the JSON, handle the

multi-instrument data (offsets/calibrations might be needed, but at minimum, structure it for analysis).

- Step 2: Signal Search Algorithm. The core task is finding radial velocity signals (periods, amplitudes). This needs to be implemented numerically (e.g., Lomb-Scargle, periodogram, or MCMC).
- Step 3: Model Fitting & Parameter Estimation. Once candidate periods are found, fit a Keplerian model to extract parameters (P, K, e, etc.).
- Step 4: Output Formatting. Structure the results into the required `agent_submission.json` format.
- Step 5: Verification/Validation. The plan needs a step that verifies the result (e.g., checking residuals, consistency, or format compliance).

3. Refine Steps for Constraints:

- *Constraint Check*: Max 5 steps.
- *Constraint Check*: Do not solve.
- *Constraint Check*: Explicit verification step.
- *Constraint Check*: No planet parameters.

Draft Plan:

1. Load `stargazer_observations.json` and preprocess time/RV data, accounting for instrument offsets if necessary.
2. Implement a periodogram search (e.g., Lomb-Scargle) to identify significant periodic signals in the radial velocity data.
3. Fit a Keplerian orbital model to the top candidate periods to estimate physical parameters (period, semi-amplitude, etc.).
4. Format the estimated parameters and metadata into the required `agent_submission.json` structure.
5. Verify the submission by checking residual noise levels and ensuring the output file matches the expected schema.

Critique: Step 5 is verification. Step 1-4 are execution. This fits the “engineer output contract” (write code that does this). The planner is telling the engineer what the code should do.

Refinement for “Planner” Role: The planner is instructing the *team* (specifically the engineer) on what the plan is. The steps should describe the *actions* the engineer’s code must perform.

Revised Plan:

1. Ingest `stargazer_observations.json` and parse the parallel arrays for time, RV, sigma, and instrument identifiers.
2. Execute a numerical period search (e.g., generalized Lomb-Scargle) across the combined dataset to detect significant periodicities.
3. Perform non-linear least-squares fitting on the top candidate periods to derive Keplerian orbital parameters.
4. Serialize the best-fit parameters and model diagnostics into `agent_submission.json` adhering to the output contract.
5. Validate the generated submission by confirming file existence, schema compliance, and residual consistency checks within the script.

Check against constraints:

- Numbered list? Yes.

1.8 8. Saeed/Qwen Engineer, Reviewer, Executor, Reviewer Loop

The engineer writes Python only. The same reviewer agent first runs in `code_review` mode. Only `APPROVE_CODE` allows saving and executing the script. After execution, the same reviewer runs in `result_review` mode and either returns `REVISE_RESULT` feedback to the engineer or approves the result.

Reviewer-setting test: this notebook now passes the planner output into both reviewer modes, keeps the same Saeed critic identity for code and result review, raises the default loop budget to 100 iterations, and adds a deadline policy. The reviewer is expected to judge scientific and numerical plausibility, but keep criteria stable across iterations and use late iterations to converge rather than oscillate on interchangeable numerical conventions. For quicker notebook experiments, the loop and deadline thresholds can be overridden with `STARGAZER_AGENT_MAX_ITERATIONS`, `STARGAZER_AGENT_DEADLINE_CONVERGENCE_START`, and `STARGAZER_AGENT_DEADLINE_FINAL_START` without changing the default 100-iteration setting.

```
[9]: FORBIDDEN_ENGINEER_TERMS = [
    "truth", "benchmark", "real_001.json", "Stargazer_real_data_task",
    ↪ "evaluator",
    "answer", "ground_truth", "stargazer" + "_inference", "infer_" +
    ↪ "stargazer_submission",
]

def strip_code_fences(text: str) -> str:
    cleaned = text.strip()
    if cleaned.startswith("```"):
        lines = cleaned.splitlines()
        if lines and lines[0].startswith("```"):
            lines = lines[1:]
        if lines and lines[-1].strip().startswith("```"):
            lines = lines[:-1]
        cleaned = "\n".join(lines).strip()
    return cleaned

def parse_review_verdict(text: str, mode: str) -> str:
    upper = text.upper()
    allowed = {"code_review": ["APPROVE_CODE", "REVISE_CODE"], "result_review":
    ↪ ["APPROVE_RESULT", "REVISE_RESULT"]}
    for verdict in allowed:
        if verdict in upper:
            return verdict
    return allowed[1]

def static_code_findings(code: str) -> list[str]:
    lower = code.lower()
```

```

findings = []
try:
    ast.parse(code)
except SyntaxError as exc:
    findings.append(f"engineer output is not valid Python: line {exc.
↳lineno}, {exc.msg}")
    if "agent_submission.json" not in code:
        findings.append("missing required agent_submission.json write")
    if "stargazer_observations.json" not in code:
        findings.append("missing required stargazer_observations.json read")
    if "def " not in code or "for " not in code:
        findings.append("code does not show enough internal numerical/search_
↳structure")
    for unsafe in ["subprocess", "os.system", "eval(", "exec(", "socket",
↳"requests", "urllib", "openai"]:
        if unsafe in lower:
            findings.append(f"unsafe or unnecessary capability in engineer code:
↳ {unsafe}")
    for term in FORBIDDEN_ENGINEER_TERMS:
        if term.lower() in lower:
            findings.append(f"forbidden reference: {term}")
return findings

def run_executor(script_path: Path, work_dir: Path) -> dict:
    before = {path.name for path in work_dir.iterdir()}
    command = [sys.executable, str(script_path)]
    result = run_command(command, cwd=work_dir, timeout=180)
    after_paths = sorted(path.name for path in work_dir.iterdir())
    generated_files = [name for name in after_paths if name not in before]
    submission_path = work_dir / "agent_submission.json"
    diagnostics_path = work_dir / "agent_diagnostics.json"
    parsed_submission = None
    parsed_diagnostics = None
    parse_error = None
    diagnostics_parse_error = None
    if submission_path.exists():
        try:
            parsed_submission = json.loads(submission_path.
↳read_text(encoding="utf-8"))
        except Exception as exc:
            parse_error = repr(exc)
    if diagnostics_path.exists():
        try:
            parsed_diagnostics = json.loads(diagnostics_path.
↳read_text(encoding="utf-8"))
        except Exception as exc:

```

```

        diagnostics_parse_error = repr(exc)
    return {
        "role": "executor",
        "command": command,
        "script_path": str(script_path),
        "cwd": str(work_dir),
        "exit_code": result["returncode"],
        "stdout": result["stdout"],
        "stderr": result["stderr"],
        "elapsed_seconds": result["elapsed_seconds"],
        "generated_files": generated_files,
        "all_files": after_paths,
        "submission_path": str(submission_path),
        "parsed_submission": parsed_submission,
        "parse_error": parse_error,
        "diagnostics_path": str(diagnostics_path),
        "parsed_diagnostics": parsed_diagnostics,
        "diagnostics_parse_error": diagnostics_parse_error,
    }

def submission_rows(submission: dict) -> list[dict]:
    planets = submission.get("planets", []) if isinstance(submission, dict)
    ↪ else []
    rows = []
    for idx, planet in enumerate(planets):
        if isinstance(planet, dict):
            rows.append({"planet_index": idx, **planet})
    return rows

transition_records = [{
    "role": "planner", "phase": "planning", "iteration": 0,
    "verdict": "PLAN_WRITTEN", "reasoning_present":
    ↪ planner_record["reasoning_present"],
    "finish_reason": planner_record["finish_reason"], "details":
    ↪ planner_summary[:240],
}]
code_review_records = []
result_review_records = []
executor_records = []
feedback = ""
engineer_code = ""
final_submission_path = None
final_submission = None
loop_stop_reason = None
max_iterations = int(os.getenv("STARGAZER_AGENT_MAX_ITERATIONS", "100"))
deadline_convergence_start = int(os.
    ↪ getenv("STARGAZER_AGENT_DEADLINE_CONVERGENCE_START", "81"))

```

```

deadline_final_start = int(os.getenv("STARGAZER_AGENT_DEADLINE_FINAL_START",
    ↪str(max(1, max_iterations - 4))))
deadline_policy = f'''
Loop budget and deadline:
- Maximum iterations: {max_iterations}.
- Iterations 1-{deadline_convergence_start - 1}: perform full scientific,
    ↪numerical, safety, and schema review.
- Iterations {deadline_convergence_start}-{deadline_final_start - 1}:
    ↪prioritize convergence; reject only for concrete blockers that would make
    ↪execution unsafe, invalid, non-physical, or unable to produce a
    ↪benchmark-ready submission.
- Iterations {deadline_final_start}-{max_iterations}: final-deadline mode. Keep
    ↪scientific standards, but do not block on stylistic preferences, debatable
    ↪parameterization choices, or interchangeable numerical conventions. If
    ↪static guardrails pass and the script implements a plan-aligned, physically
    ↪plausible internal search, approve code for execution so result_review can
    ↪inspect empirical output.
- On the final iteration, return the best terminal verdict available for the
    ↪current mode. Do not invent a new criterion that contradicts earlier
    ↪feedback.
'''

def flush_agent_loop_logs():
    write_json(OUT / "qwen_trace_full.json", qwen_trace)
    write_json(OUT / "code_review_decisions.json", code_review_records)
    write_json(OUT / "result_review_decisions.json", result_review_records)
    write_json(OUT / "executor_records.json", executor_records)
    write_json(OUT / "agent_transition_trace.json", transition_records)

base_engineer_prompt = f'''
Planner output:
{planner_summary}

Write a complete Python script for the STARGAZER finding task.

Hard contract:
- Output Python code only. No markdown fences and no prose.
- The code must read stargazer_observations.json from the current working
    ↪directory.
- The input JSON is a top-level dict with task_id and observations;
    ↪observations is a dict of parallel arrays named times_days, rvs_ms,
    ↪sigmas_ms, and instruments. If the observations key exists, read arrays from
    ↪data["observations"].
- The code must write agent_submission.json in the current working directory.

```

- The code must not read truth, benchmark, task, answer, evaluator, or hidden_ files.
- Do not include the forbidden file or validation words anywhere in the script, including comments or strings: truth, benchmark, ground_truth, evaluator, answer, real_001.json, Stargazer_real_data_task, " + "stargazer" + "_inference" + ", " + "infer_" + "stargazer_submission" + ".\n",
- The code must not import or call any prebuilt STARGAZER inference module.
- The code should implement its own numerical search logic using only common Python scientific packages if available, with standard-library fallbacks.
- For radial-velocity planet finding, the numerical search should test a broad astrophysically plausible period range, compare multiple top candidates, and guard against daily/sub-day aliases. Do not blindly accept a sub-day period unless the code demonstrates it is decisively better than nearby daily aliases and multi-day candidates.
- Candidate selection should use a quantitative fit criterion such as weighted residual improvement, reduced chi-square, or BIC/AIC across the top period candidates, not just the single largest raw periodogram peak.
- When converting RV semi-amplitude K to m_sin_i_mjup, use SI units carefully: period in seconds, K in m/s, solar-mass stellar assumption unless otherwise stated, and sanity-check that tens of m/s at a few-day period gives a sub-Jupiter to order-one-Jupiter mass rather than many Jupiter masses.
- The script may write agent_diagnostics.json with non-secret diagnostics such as top candidate periods, scores, residual metrics, and alias checks to help result_review judge scientific plausibility.
- The JSON output should be a dict with a planets list. Each planet should include P_days, m_sin_i_mjup, e, omega_rad, and l_rad when estimated.

```
'''
for iteration in range(1, max_iterations + 1):
    engineer_prompt = base_engineer_prompt if not feedback else f'''
Revise the previous script using the reviewer/executor feedback below.
```

Previous script:
{engineer_code}

Feedback:
{feedback}

Return a full corrected Python script only.

```
'''
    engineer_record = qwen_role_call("engineer", ENGINEER_SYSTEM_MESSAGE,
    engineer_prompt, max_tokens=6000, phase="engineering", iteration=iteration,
    enable_thinking=False)
    engineer_code = strip_code_fences(engineer_record["text"])
    transition_records.append({
        "role": "engineer", "phase": "write_code", "iteration": iteration,
```



```

        "verdict": "SCRIPT_PROPOSED", "reasoning_present":  

        ↪engineer_record["reasoning_present"],  

        "finish_reason": engineer_record["finish_reason"], "details":  

        ↪f"{len(engineer_code)} characters",  

        })  
  

        static_findings = static_code_findings(engineer_code)  

        code_review_prompt = f'''  

Mode: code_review  
  

Review the engineer script before execution. Use the Saeed critic/reviewer  

        ↪identity.  
  

Planner output that the reviewer must use as review evidence:  

{planner_summary}  
  

{deadline_policy}  
  

Current iteration: {iteration} of {max_iterations}.  
  

Return exactly one verdict line first:  

- APPROVE_CODE if the script is safe to execute and can plausibly produce  

        ↪agent_submission.json.  

- REVISE_CODE if the engineer must revise before execution.  
  

Check for code safety, output schema, no hidden truth files, no benchmark  

        ↪files, no external precomputed inference modules, and plausible internal  

        ↪numerical search logic.  

Also judge physical and numerical validity: whether the search/fitting approach  

        ↪is scientifically plausible for radial-velocity planet finding, whether  

        ↪units and output fields are reasonable, and whether the code can plausibly  

        ↪recover periodic signals. Keep this standard stable across iterations.  
  

Static findings from notebook guardrail:  

{json.dumps(static_findings, indent=2)}  
  

Engineer script:  

{engineer_code}  

'''  

        code_review_record = qwen_role_call("reviewer", REVIEWER_SYSTEM_MESSAGE,  

        ↪code_review_prompt, max_tokens=1200, phase="code_review",  

        ↪iteration=iteration, enable_thinking=False)  

        code_verdict = parse_review_verdict(code_review_record["text"],  

        ↪"code_review")  

        if static_findings:  

            code_verdict = "REVISE_CODE"

```

```

code_review_payload = {**code_review_record, "mode": "code_review",
↪ "verdict": code_verdict, "static_findings": static_findings}
code_review_records.append(code_review_payload)
transition_records.append({
    "role": "reviewer", "phase": "code_review", "iteration": iteration,
    "verdict": code_verdict, "reasoning_present":
↪ code_review_record["reasoning_present"],
    "finish_reason": code_review_record["finish_reason"], "details":
↪ code_review_record["text"][:240],
})
flush_agent_loop_logs()

if code_verdict == "REVISE_CODE":
    feedback = "Code review requires revision. " +
↪ code_review_record["text"] + "\nStatic findings: " + json.
↪ dumps(static_findings)
    loop_stop_reason = "max_iterations" if iteration == max_iterations else
↪ None
    if not loop_stop_reason:
        transition_records.append({
            "role": "reviewer", "phase": "feedback_to_engineer",
↪ "iteration": iteration,
            "verdict": "REVISE_CODE", "reasoning_present":
↪ code_review_record["reasoning_present"],
            "finish_reason": code_review_record["finish_reason"], "details":
↪ "code feedback returned before next engineer iteration",
        })
        flush_agent_loop_logs()
    if loop_stop_reason:
        flush_agent_loop_logs()
        break
    continue

iteration_dir = AGENT_WORK / f"iteration_{iteration:02d}"
iteration_dir.mkdir(parents=True, exist_ok=True)
shutil.copy2(AGENT_OBS_COPY, iteration_dir / "stargazer_observations.json")
script_path = iteration_dir / f"engineer_iteration_{iteration:02d}.py"
script_path.write_text(engineer_code, encoding="utf-8")
transition_records.append({
    "role": "reviewer", "phase": "save_approved_code", "iteration":
↪ iteration,
    "verdict": "APPROVE_CODE", "reasoning_present":
↪ code_review_record["reasoning_present"],
    "finish_reason": code_review_record["finish_reason"], "details":
↪ str(script_path),
})

```

```

flush_agent_loop_logs()

executor_record = run_executor(script_path, iteration_dir)
executor_records.append(executor_record)
write_json(iteration_dir / "executor_record.json", executor_record)
transition_records.append({
    "role": "executor", "phase": "execute", "iteration": iteration,
    "verdict": "EXECUTED", "reasoning_present": False,
    "finish_reason": None, "details":
    ↪f"exit_code={executor_record['exit_code']}",
    })
flush_agent_loop_logs()

result_review_prompt = f'''
Mode: result_review

Review the executor logs and produced submission. Use the same Saeed critic/
    ↪reviewer identity.

Planner output that the reviewer must use as review evidence:
{planner_summary}

{deadline_policy}

Current iteration: {iteration} of {max_iterations}.

Return exactly one verdict line first:
- APPROVE_RESULT if the executor succeeded, agent_submission.json is valid, the
    ↪planet table is plausible enough for benchmark, and the result is ready for
    ↪separate STARGAZER scoring.
- REVISE_RESULT if the engineer must revise code and try again.

Judge scientific plausibility of the executed result: non-empty planet table,
    ↪reasonable periods/amplitudes/eccentricities/angles, no obvious unit
    ↪mistakes, and consistency with the planner's intended radial-velocity search
    ↪workflow. Reject suspicious sub-day or near-daily-alias periods unless
    ↪diagnostics show they decisively beat broad multi-day candidates under a
    ↪weighted fit criterion. Check K-to-mass consistency: for a solar-mass star,
    ↪K near tens of m/s at a few-day period implies a sub-Jupiter to
    ↪order-one-Jupiter mass, not many Jupiter masses. Reject if diagnostics and
    ↪submission imply inconsistent K, mass, period, or units. Also reject if the
    ↪selected model has very poor weighted residuals or BIC compared with
    ↪alternatives, even when the period is plausible. Prefer result revisions
    ↪that ask the engineer to broaden the period search, compare top candidates,
    ↪improve instrument-offset treatment, fix mass conversion, include
    ↪stellar-mass assumptions, or emit diagnostics. The hidden truth remains
    ↪unavailable here; benchmark scoring happens only after approval.

```

Executor record:

```
{json.dumps(executor_record, indent=2)[:12000]}
'''

    result_review_record = qwen_role_call("reviewer", REVIEWER_SYSTEM_MESSAGE,
↪result_review_prompt, max_tokens=1200, phase="result_review",
↪iteration=iteration, enable_thinking=False)
    result_verdict = parse_review_verdict(result_review_record["text"],
↪"result_review")
    parsed = executor_record.get("parsed_submission")
    if executor_record["exit_code"] != 0 or not isinstance(parsed, dict) or not
↪isinstance(parsed.get("planets"), list) or not parsed.get("planets"):
        result_verdict = "REVISE_RESULT"
    result_review_payload = {**result_review_record, "mode": "result_review",
↪"verdict": result_verdict}
    result_review_records.append(result_review_payload)
    transition_records.append({
        "role": "reviewer", "phase": "result_review", "iteration": iteration,
        "verdict": result_verdict, "reasoning_present":
↪result_review_record["reasoning_present"],
        "finish_reason": result_review_record["finish_reason"], "details":
↪result_review_record["text"][:240],
    })
    flush_agent_loop_logs()

    if isinstance(parsed, dict):
        final_submission_path = Path(executor_record["submission_path"])
        final_submission = parsed

    if result_verdict == "APPROVE_RESULT":
        loop_stop_reason = "APPROVE_RESULT"
        flush_agent_loop_logs()
        break

    feedback = "Result review requires revision. " +
↪result_review_record["text"] + "\nExecutor record: " + json.
↪dumps(executor_record, indent=2)[:4000]
    loop_stop_reason = "max_iterations" if iteration == max_iterations else None
    if not loop_stop_reason:
        transition_records.append({
            "role": "reviewer", "phase": "feedback_to_engineer", "iteration":
↪iteration,
            "verdict": "REVISE_RESULT", "reasoning_present":
↪result_review_record["reasoning_present"],
            "finish_reason": result_review_record["finish_reason"], "details":
↪"result feedback returned before next engineer iteration",
```

```

    })
    flush_agent_loop_logs()

if loop_stop_reason is None:
    loop_stop_reason = "max_iterations"

write_json(OUT / "qwen_trace_full.json", qwen_trace)
write_json(OUT / "code_review_decisions.json", code_review_records)
write_json(OUT / "result_review_decisions.json", result_review_records)
write_json(OUT / "executor_records.json", executor_records)
write_json(OUT / "agent_transition_trace.json", transition_records)

transition_df = pd.DataFrame(transition_records)
display(transition_df[["role", "phase", "iteration", "verdict",
    ↪ "reasoning_present", "finish_reason", "details"]])

def next_engineer_exists_after(record: dict) -> bool:
    return any(
        later["role"] == "engineer" and later["phase"] == "write_code" and
        ↪ later["iteration"] > record["iteration"]
        for later in transition_records
    )

assert transition_records[0]["role"] == "planner"
assert transition_records[1]["role"] == "engineer"
assert transition_records[2]["role"] == "reviewer" and
    ↪ transition_records[2]["phase"] == "code_review"
assert all(r["phase"] != "execute" or any(x["iteration"] == r["iteration"] and
    ↪ x["verdict"] == "APPROVE_CODE" for x in transition_records) for r in
    ↪ transition_records)
assert all(r["phase"] != "result_review" or any(x["iteration"] ==
    ↪ r["iteration"] and x["phase"] == "execute" for x in transition_records) for
    ↪ r in transition_records)
assert all(next_engineer_exists_after(r) for r in transition_records if
    ↪ r["phase"] == "feedback_to_engineer" and r["verdict"] in ["REVISE_CODE",
    ↪ "REVISE_RESULT"])
assert all(Path(r["script_path"]).exists() and r["command"][1] ==
    ↪ r["script_path"] for r in executor_records)
assert all(Path(r["submission_path"]).parent == Path(r["cwd"]) for r in
    ↪ executor_records)
assert loop_stop_reason in ["APPROVE_RESULT", "max_iterations"]
if loop_stop_reason == "APPROVE_RESULT":
    assert final_submission_path is not None and final_submission_path.
    ↪ exists(), "No executor-produced final submission exists."

```

```

    role                phase  iteration                verdict
    ↪ reasoning_present finish_reason \

```

0	planner	planning	0	PLAN_WRITTEN	␣
	↪True	length			
1	engineer	write_code	1	SCRIPT_PROPOSED	␣
	↪False	stop			
2	reviewer	code_review	1	REVISE_CODE	␣
	↪False	stop			
3	reviewer	feedback_to_engineer	1	REVISE_CODE	␣
	↪False	stop			
4	engineer	write_code	2	SCRIPT_PROPOSED	␣
	↪False	stop			
5	reviewer	code_review	2	REVISE_CODE	␣
	↪False	stop			
6	reviewer	feedback_to_engineer	2	REVISE_CODE	␣
	↪False	stop			
7	engineer	write_code	3	SCRIPT_PROPOSED	␣
	↪False	stop			
8	reviewer	code_review	3	REVISE_CODE	␣
	↪False	stop			
9	reviewer	feedback_to_engineer	3	REVISE_CODE	␣
	↪False	stop			
10	engineer	write_code	4	SCRIPT_PROPOSED	␣
	↪False	stop			
11	reviewer	code_review	4	REVISE_CODE	␣
	↪False	length			
12	reviewer	feedback_to_engineer	4	REVISE_CODE	␣
	↪False	length			
13	engineer	write_code	5	SCRIPT_PROPOSED	␣
	↪False	stop			
14	reviewer	code_review	5	APPROVE_CODE	␣
	↪False	stop			
15	reviewer	save_approved_code	5	APPROVE_CODE	␣
	↪False	stop			
16	executor	execute	5	EXECUTED	␣
	↪False	NaN			
17	reviewer	result_review	5	REVISE_RESULT	␣
	↪False	stop			
18	reviewer	feedback_to_engineer	5	REVISE_RESULT	␣
	↪False	stop			
19	engineer	write_code	6	SCRIPT_PROPOSED	␣
	↪False	stop			
20	reviewer	code_review	6	APPROVE_CODE	␣
	↪False	stop			
21	reviewer	save_approved_code	6	APPROVE_CODE	␣
	↪False	stop			
22	executor	execute	6	EXECUTED	␣
	↪False	NaN			

23	reviewer	result_review	6	REVISE_RESULT	□
	↪False	stop			
24	reviewer	feedback_to_engineer	6	REVISE_RESULT	□
	↪False	stop			
25	engineer	write_code	7	SCRIPT_PROPOSED	□
	↪False	stop			
26	reviewer	code_review	7	APPROVE_CODE	□
	↪False	stop			
27	reviewer	save_approved_code	7	APPROVE_CODE	□
	↪False	stop			
28	executor	execute	7	EXECUTED	□
	↪False	NaN			
29	reviewer	result_review	7	APPROVE_RESULT	□
	↪False	stop			

↪ □

↪ □

↪details

0 Thinking Process:\n\n1. **Analyze the Request:**\n * **Role:** PLANNER, □
↪in a multi-agent scientific-reasoning team.\n * **Task:** Create a short, □
↪...

1 □

↪ 6430 □

↪characters

2 VERDICT: REVISE_CODE - The `run\_periodogram` function contains a `pass` □
↪statement inside the manual Lomb-Scargle loop that prevents the calculation of □
↪`powe...

3 □

↪ code feedback returned before next engineer □

↪iteration

4 □

↪ 5549 □

↪characters

5 VERDICT: REVISE_CODE - The `run\_periodogram` function uses `scipy.signal. □
↪lombscargle` with `weights` set to inverse variance, but this function □
↪signature do...

6 □

↪ code feedback returned before next engineer □

↪iteration

7 □

↪ 5728 □

↪characters

8 VERDICT: REVISE_CODE - The `run\_periodogram` function incorrectly normalizes □
↪the radial velocity data by dividing by `sigma` before passing to □
↪`lombscargle`...

```

9
    ↪ code feedback returned before next engineer
    ↪ iteration
10
    ↪ 6791
    ↪ characters
11 VERDICT: REVISE_CODE - The `run_periodogram` function incorrectly passes
    ↪ `weights` to `scipy.signal.lombscargle`, which does not support a `weights`
    ↪ argumen...
12
    ↪ code feedback returned before next engineer
    ↪ iteration
13
    ↪ 5760
    ↪ characters
14 VERDICT: APPROVE_CODE\n\nThe script implements the planned workflow
    ↪ correctly:\n1. **Data Loading**: Correctly parses the JSON structure and
    ↪ extracts array...
15 C:
    ↪ \Users\Anwender\Science-Work-Flow-\outputs\qwen_saeed_agent_stargazer\agent_workflow\iterati
    ↪ py
16
    ↪
    ↪ exit_code=0
17 VERDICT: REVISE_RESULT - The agent_submission.json contains an empty planet
    ↪ table, failing the requirement for a non-empty result with plausible orbital
    ↪ par...
18
    ↪ result feedback returned before next engineer
    ↪ iteration
19
    ↪ 7819
    ↪ characters
20 The engineer's script implements the planned workflow correctly:\n1. **Data
    ↪ Loading**: Correctly parses `stargazer_observations.json` and handles the
    ↪ multi...
21 C:
    ↪ \Users\Anwender\Science-Work-Flow-\outputs\qwen_saeed_agent_stargazer\agent_workflow\iterati
    ↪ py
22
    ↪
    ↪ exit_code=0
23 VERDICT: REVISE_RESULT - The reported semi-amplitude (K 57.9 m/s) combined
    ↪ with a circular orbit (e=0) yields a physically impossible mass (m sin i 3.3
    ↪ ...

```



```

24                                     result feedback returned before next engineer
↳ iteration
25                                     7427
↳ characters
26 The engineer's script implements the planned workflow correctly and safely.
↳\n\n1.  **Safety & Compliance**: The script uses standard scientific libraries
↳(`...
27                                     C:
↳\Users\Anwender\Science-Work-Flow-\outputs\qwen_saeed_agent_stargazer\agent_workflow\iterati
↳py
28
↳ exit_code=0
29 The executor produced a valid `agent_submission.json` with a non-empty
↳planet table. The selected planet has a period of ~4.23 days and a
↳semi-amplitude of ...

```

1.9 9. Clean Executor-Produced Finding Results

This table is built only from `agent_submission.json` produced by the executor. It is not a benchmark judgment and it is not generated from hidden truth.

```

[10]: result_rows = submission_rows(final_submission)
result_table = pd.DataFrame(result_rows)
display(result_table)
write_json(OUT / "clean_star_finding_results.json", result_rows)

clean_summary = {
    "target_star": "real_001",
    "finding_task": True,
    "submitted_planet_count": len(result_rows),
    "final_submission_path": str(final_submission_path),
    "qwen_provider_ok": probe["ok"],
    "qwen_thinking_enabled": True,
    "saeed_planner_output": bool(planner_summary.strip()),
    "saeed_engineer_output": bool(engineer_code.strip()),
    "reviewer_code_reviews": len(code_review_records),
    "reviewer_result_reviews": len(result_review_records),
    "loop_stop_reason": loop_stop_reason,
    "hidden_truth_used_for_agent_workflow": False,
}
write_json(OUT / "clean_star_finding_summary.json", clean_summary)
display(pd.DataFrame([[k, v] for k, v in clean_summary.items()]),
↳columns=["Field", "Value"])

```

	planet_index	P_days	m_sin_i_mjup	e	omega_rad	l_rad
0	0	4.232078	4.390132	2.699629e-15	0.0	6.283185
			Field			
0			target_star			
1			finding_task			
2			submitted_planet_count			
3			final_submission_path	C:		
4			qwen_provider_ok			
5			qwen_thinking_enabled			
6			saeed_planner_output			
7			saeed_engineer_output			
8			reviewer_code_reviews			
9			reviewer_result_reviews			
10			loop_stop_reason			
11			hidden_truth_used_for_agent_workflow			

1.10 10. Separate STARGAZER Benchmark Judgment

This benchmark section is intentionally separate from the agent workflow. It runs only after `reviewer(result_review)` returns `APPROVE_RESULT` for the executor-produced `agent_submission.json`, and the hidden-truth task file is used only inside the benchmark call.

```
[11]: from cambagent_eval.stargazer_minimal import evaluate_stargazer_benchmark

benchmark_ran = loop_stop_reason == "APPROVE_RESULT"
if benchmark_ran:
    assert final_submission_path is not None and final_submission_path.
    exists(), "Benchmark requires final executor-produced submission."
    assert str(final_submission_path).startswith(str(AGENT_WORK)), "Benchmark_
    must read the executor-produced submission from the agent workflow directory.
    "

    task_json = SCIENCE_ROOT / "data" / "stargazer_repo" / "stargazer" /
    "Stargazer_real_data_task" / "real_001.json"
    benchmark = evaluate_stargazer_benchmark(final_submission_path, task_json)
else:
    benchmark = {
        "skipped": True,
        "reason": "reviewer(result_review) did not return APPROVE_RESULT before_
        max_iterations",
        "loop_stop_reason": loop_stop_reason,
        "final_submission_path": str(final_submission_path) if
        final_submission_path else None,
    }
write_json(OUT / "separate_stargazer_benchmark_judgment.json", benchmark)

benchmark_rows = [
    ["benchmark_ran", benchmark_ran],
    ["benchmark_skipped", benchmark.get("skipped", False)],
    ["skip_reason", benchmark.get("reason")],
    ["benchmark_evaluable", benchmark.get("evaluable")],
    ["benchmark_passed", benchmark.get("passed")],
    ["score", benchmark.get("score")],
    ["reward", benchmark.get("reward")],
    ["match_score", benchmark.get("match_score")],
    ["matched_truth_fraction", benchmark.get("matched_truth_fraction")],
    ["rms", benchmark.get("rms")],
    ["submitted_planet_count", benchmark.get("submitted_planet_count")],
    ["truth_planet_count", benchmark.get("truth_planet_count")],
]
display(pd.DataFrame(benchmark_rows, columns=["Benchmark field", "Value"]))
display(pd.DataFrame(benchmark.get("prediction_truth_rows", [])))
```

	Benchmark field	Value
0	benchmark_ran	True
1	benchmark_skipped	False
2	skip_reason	None
3	benchmark_evaluable	True
4	benchmark_passed	False

```

5         score          0.4
6         reward -335387.464489
7         match_score      0.0
8  matched_truth_fraction  0.0
9         rms      354.505911
10  submitted_planet_count  1
11  truth_planet_count     1

      row_type  truth_index  guess_index      truth \
↪
0  unmatched_truth      0.0      NaN  {'P_days': 4.230785, ↪
↪ 'm_sin_i_mjup': 0.461, 'e': 0.013, 'omega_rad': 1.012291, 'l_rad': 4.644516}
1  unmatched_submission      NaN      0.0      {}
↪

↪      submission
0
↪      {}
1  {'P_days': 4.232078, 'm_sin_i_mjup': 4.390132, 'e': 0.0, 'omega_rad': 0.0, ↪
↪ 'l_rad': 6.283185}

```

1.11 11. Confirm Saeed Repo Was Not Modified

```

[12]: post_status = run_command(["git", "status", "--short"], cwd=SAEED_REPO)
post_audit = {
    "pre_status_short": preflight["status_short"],
    "post_status_short": post_status["stdout"].strip(),
    "repo_untouched": preflight["status_short"] == post_status["stdout"].
↪ strip() == "",
}
(OUT / "finding_repo_post_audit.json").write_text(json.dumps(post_audit, ↪
↪ indent=2), encoding="utf-8")
display(pd.DataFrame([[k, v] for k, v in post_audit.items()], columns=["Field", ↪
↪ "Value"]))
assert post_audit["repo_untouched"], "Saeed repo changed."

```

```

      Field Value
0  pre_status_short
1  post_status_short
2    repo_untouched  True

```

1.12 12. Final Verdict

This notebook now separates three evidence layers:

- sanitized observations and planner context;
- live Saeed/Qwen planner-engineer-reviewer-executor workflow that produces agent_submission.json;

- separate STARGAZER benchmark scoring that reads only the executor-produced submission.

```
[13]: final_verdict = {
    "notebook_complete": True,
    "task_type": "real_stargazer_finding",
    "target_star": "real_001",
    "submitted_planet_count": len(result_rows),
    "final_submission_path": str(final_submission_path),
    "qwen_provider_ok": probe["ok"],
    "qwen_thinking_enabled": True,
    "loop_stop_reason": loop_stop_reason,
    "code_review_count": len(code_review_records),
    "result_review_count": len(result_review_records),
    "benchmark_ran": benchmark_ran,
    "benchmark_skipped": benchmark.get("skipped", False),
    "benchmark_evaluable": benchmark.get("evaluable"),
    "benchmark_passed": benchmark.get("passed"),
    "benchmark_match_score": benchmark.get("match_score"),
    "saeed_repo_untouched": post_audit["repo_untouched"],
    "outputs": str(OUT),
}
write_json(OUT / "finding_final_verdict.json", final_verdict)
print(json.dumps(final_verdict, indent=2))
assert final_verdict["notebook_complete"]
assert final_verdict["task_type"] == "real_stargazer_finding"
assert final_verdict["loop_stop_reason"] in ["APPROVE_RESULT", "max_iterations"]
assert final_verdict["benchmark_ran"] == (final_verdict["loop_stop_reason"] ==
    ↳ "APPROVE_RESULT")
assert final_verdict["saeed_repo_untouched"]
```

```
{
  "notebook_complete": true,
  "task_type": "real_stargazer_finding",
  "target_star": "real_001",
  "submitted_planet_count": 1,
  "final_submission_path": "C:\\Users\\Anwender\\Science-Work-Flow-
\\outputs\\qwen_saeed_agent_stargazer\\agent_workflow\\iteration_07\\agent_submi
ssion.json",
  "qwen_provider_ok": true,
  "qwen_thinking_enabled": true,
  "loop_stop_reason": "APPROVE_RESULT",
  "code_review_count": 7,
  "result_review_count": 3,
  "benchmark_ran": true,
  "benchmark_skipped": false,
  "benchmark_evaluable": true,
  "benchmark_passed": false,
  "benchmark_match_score": 0.0,
```

```

"saeed_repo_untouched": true,
"outputs": "C:\\Users\\Anwender\\Science-Work-
Flow\\outputs\\qwen_saeed_agent_stargazer"
}

```

1.13 13. Notebook Source Guardrail Checks

These checks verify the notebook source no longer contains the removed pre-agent inference path and that the role transition contract is encoded in the saved trace.

```

[14]: notebook_source = (SCIENCE_ROOT / "notebooks" / "qwen_saeed_agent_stargazer.
↳ipynb").read_text(encoding="utf-8")
removed_module_name = "stargazer" + "_inference"
removed_function_name = "infer_" + "stargazer_submission"
old_table_name = "planets" + "_df"
assert removed_module_name not in notebook_source
assert removed_function_name not in notebook_source
assert old_table_name not in notebook_source

phases = [(r["role"], r["phase"], r["verdict"]) for r in transition_records]
assert phases[0][0] == "planner"
assert phases[1][0] == "engineer"
assert phases[2][0] == "reviewer" and phases[2][1] == "code_review"
has_executor = any(p[0] == "executor" and p[1] == "execute" for p in phases)
has_approved_code = any(p[0] == "reviewer" and p[1] == "save_approved_code" and
↳p[2] == "APPROVE_CODE" for p in phases)
assert has_executor == has_approved_code
assert all(p[1] != "execute" or any(q[1] == "save_approved_code" and q[2] ==
↳"APPROVE_CODE" for q in phases) for p in phases)
assert all(p[1] != "result_review" or has_executor for p in phases)
assert all(next_engineer_exists_after(r) for r in transition_records if
↳r["phase"] == "feedback_to_engineer" and r["verdict"] in ["REVISE_CODE",
↳"REVISE_RESULT"])
assert all(Path(r["script_path"]).exists() and r["command"][1] ==
↳r["script_path"] for r in executor_records)
assert all(Path(r["submission_path"]).parent == Path(r["cwd"]) for r in
↳executor_records)
if final_submission_path is not None:
    assert final_submission_path.exists()
    assert final_submission_path.name == "agent_submission.json"
print("Notebook source and role-transition guardrails passed.")

```

Notebook source and role-transition guardrails passed.